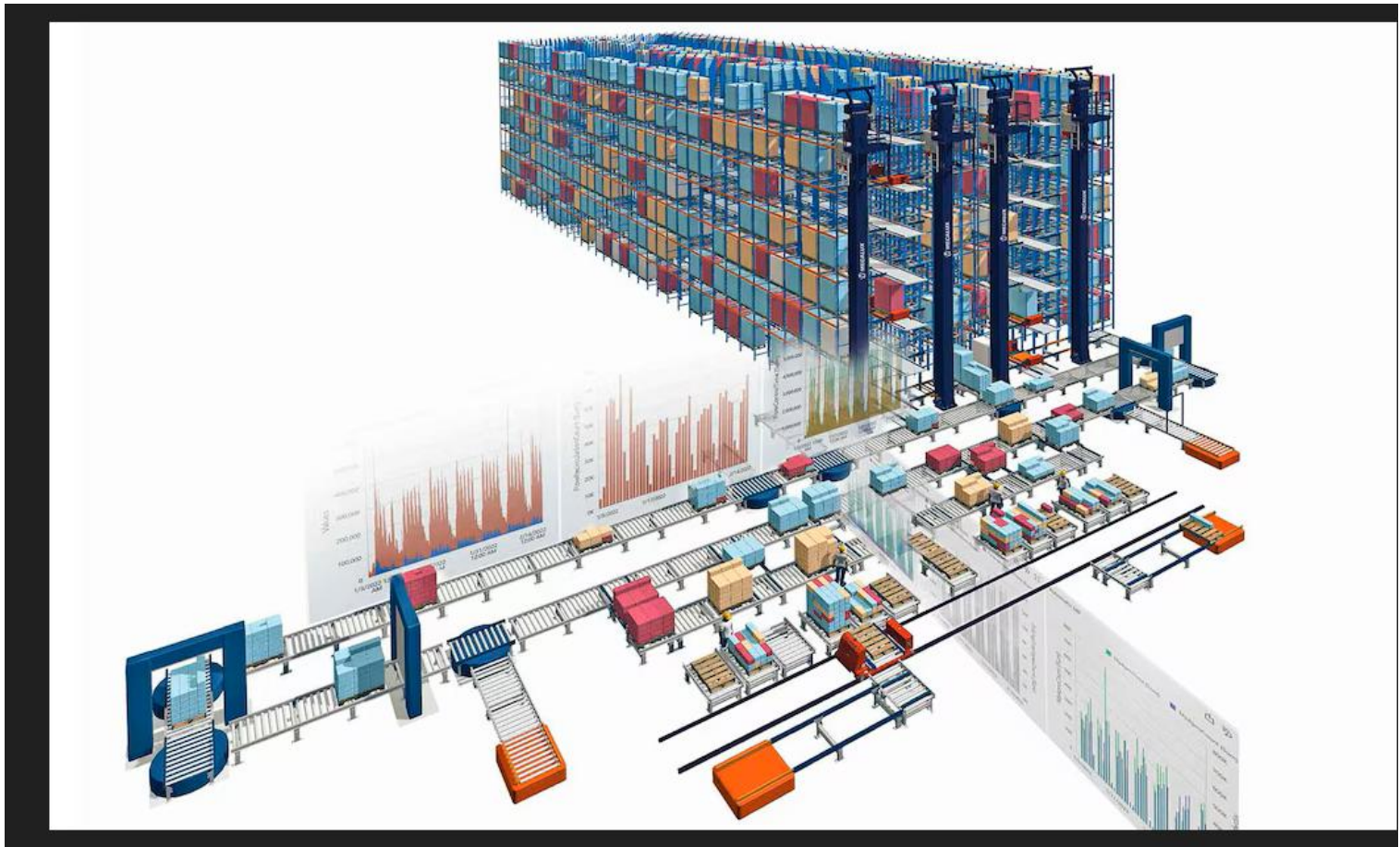


MODULE : ALGORITHMES AVANCES



Dr SADA ANNE

Chapitre 3: Arbres

.1 Introduction

- Dans les tableaux nous avons :
 - + Un accès direct par indice (rapide)
 - L'insertion et la suppression nécessitent des décalages
- Dans les listes linéaires chaînées nous avons :
 - + L'insertion et la suppression se font uniquement par modification de chaînage.
 - Accès séquentiel lent.
- Les arbres représentent un compromis entre les deux :
 - + Un accès relativement rapide à un élément.
 - + Ajout et suppression non coûteuses.

1.1 Introduction

La structure d'arbre est l'une des plus importantes et des plus spécifiques en informatique et elle permet d'écrire des algorithmes très performants.

En plus plusieurs traitements en informatique sont de nature arborescente tel que:

- L'organisation des fichiers dans les systèmes d'exploitation,
- La représentation des programmes traités par un ordinateur,
- La représentation d'une table des matières

1.2. Définition

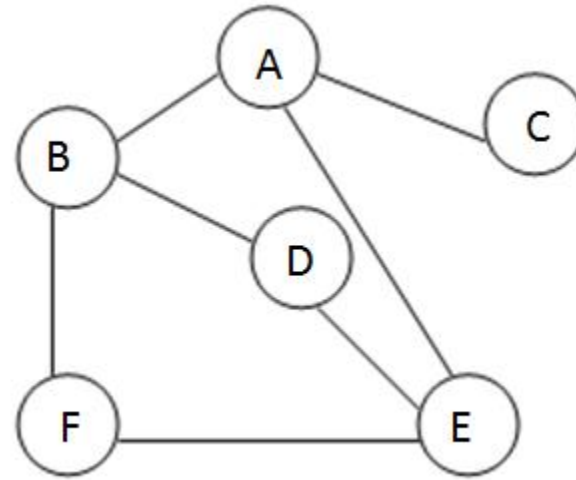
Un graphe: Un graphe $G = (S, A)$ est défini par :

- Un ensemble de sommets **S**.
- un ensemble d'arêtes **A**.

Exemples de graphes :

(villes, routes),

(machines, réseaux),

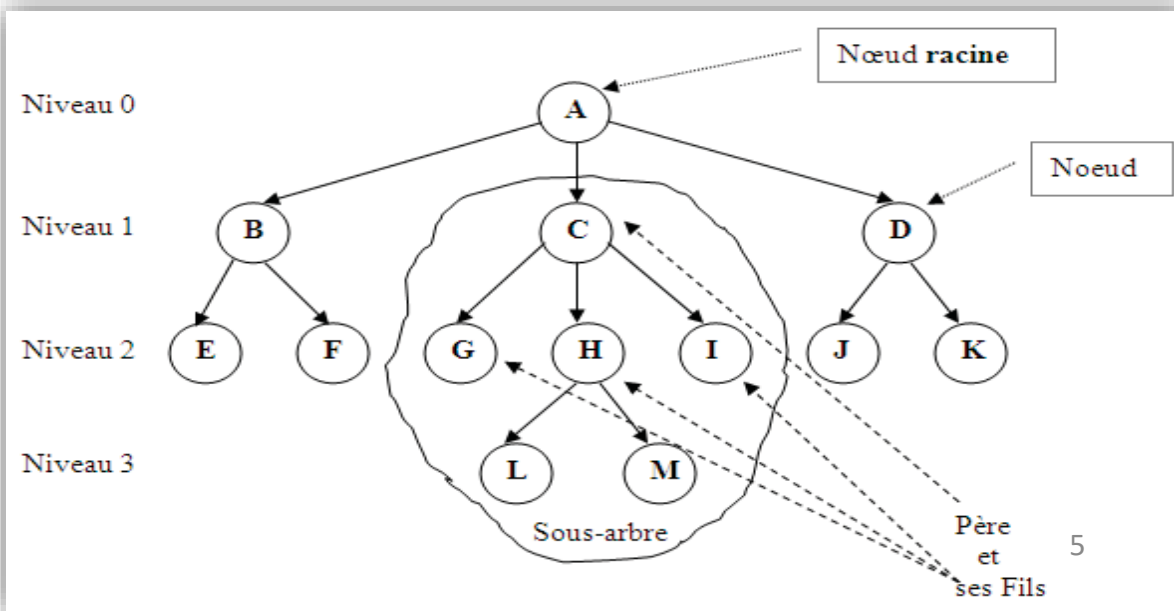


Définition:

Un arbre est une structure non linéaire, c'est un **graphe** sans cycle où chaque nœud **a au plus un prédécesseur**.

1.2. Définition

- Le **prédécesseur** s'il existe s'appelle **père** (père de C = A, père de L = H)
- Le **successeur** s'il existe s'appelle **fils** (fils de A = { B,C,D }, fils de H = { L,M })
- Le nœud qui n'a pas de prédécesseur s'appelle **racine** (A)
- Le nœud qui n'a pas de successeur s'appelle **feuille** (E,F,G,L,J,...)
- **Descendants** : de C={G,H,I,L,M}, de B={E,F},...
- **Ascendants** de L={H,C,A }, E={B,A},...



1.3 Mesures sur les arbres

Taille d'un arbre : C'est le nombre de nœuds qu'il possède.

- Taille de l'arbre précédent = 13
- Un arbre vide est de taille égale à 0.

Niveau d'un nœud : Le niveau d'un nœud est le nombre de nœuds entre celui-ci et la racine.

- Le niveau de la racine = 0
- Le niveau de chaque nœud est égale au niveau de son père plus 1
- Niveau de E,F,G,H,I,J,K = 2

Hauteur (Profondeur) d'un arbre: C'est le niveau maximum dans cet arbre.

- Hauteur de l'arbre précédent = 3

Degré d'un nœud: Le degré d'un nœud est égal au nombre de ses fils.

- Degré de (A = 3, B = 2, C = 3, E = 0, H = 2,...)

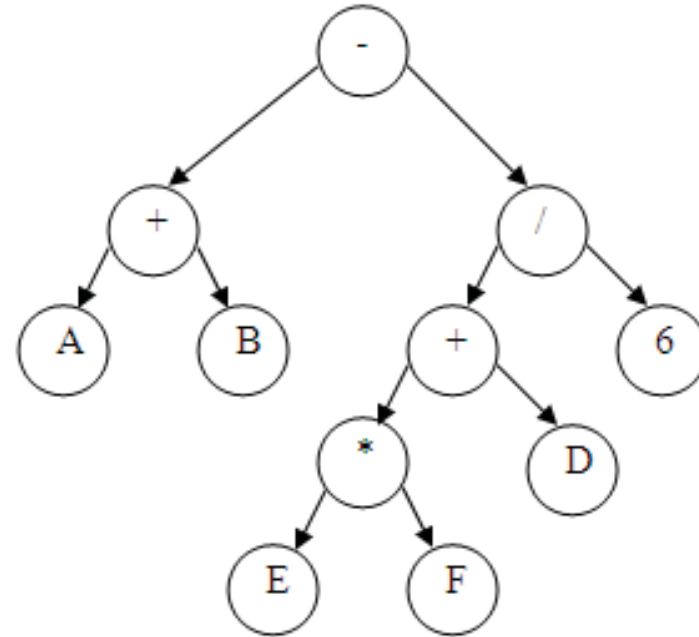
Degré d'un arbre: C'est le degré maximum de ses nœuds.

- Degré de l'arbre précédent = 3.

1.4 Exemples d'utilisation des arbres

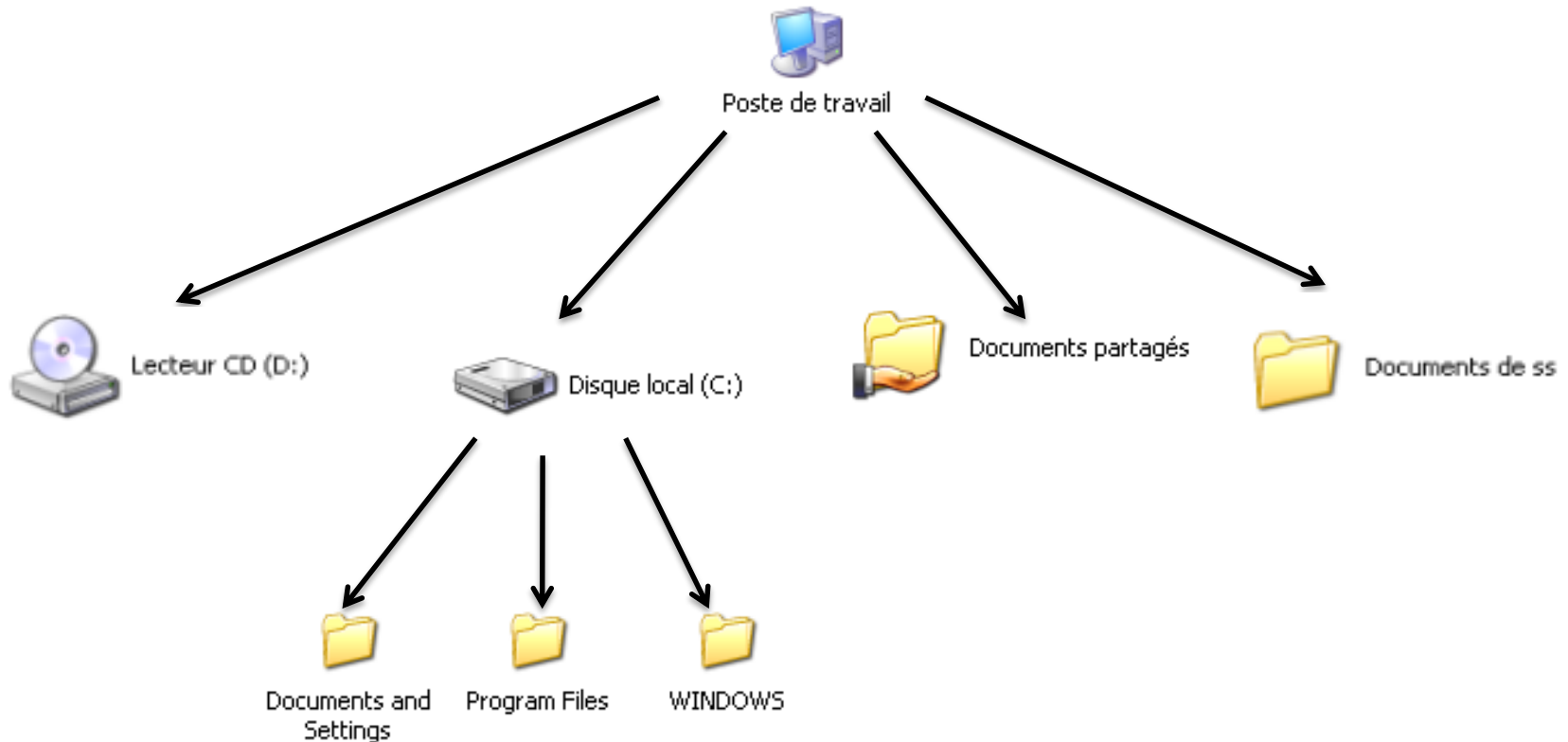
Représentation des expressions arithmétiques:

$(A+B) - (E*f + d)/6$



1.4 Exemples d'utilisation des arbres

Représentation des fichiers dans Windows.

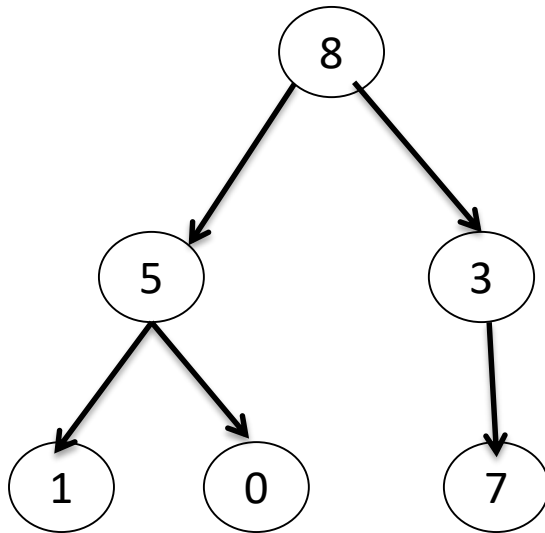


1.5 Implémentation

Les arbres peuvent être représentés par des tableaux ou des listes chinées non linéaires :

a) Représentation statique

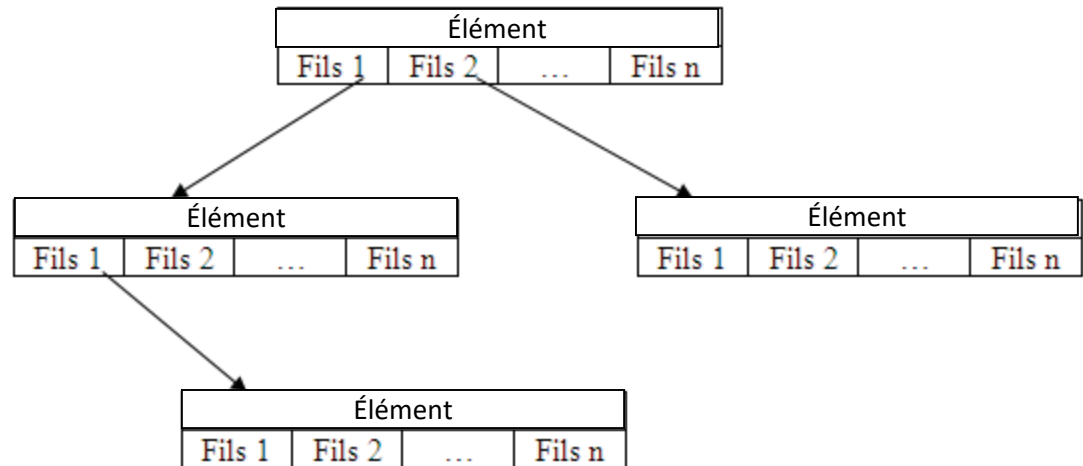
Ci-dessous la représentation d'un arbre d'entiers par un tableau:



	élément	Fils 1	Fils 2
0	8	1	2
1	5	3	4
2	3	5	-1
3	1	-1	-1
4	0	-1	-1
5	7	-1	-1

1.5 Implémentation

- b) **Représentation dynamique**: utilisation d'une liste non linéaire où chaque nœud contient un élément avec la liste (Un tableau ou une liste chaînée) de ses fils.



Définition de type

Type **Nœud** : Enregistrement

Élément : typeqq ;

Fils : Tableau[NbFils] de (*** Nœud**) ;

Fin ;

Type Arbre : ***** Nœud ;

1.5 Typologie des arbres

Arbre n-aire : un arbre m-aire d'ordre n est un arbre où le degré maximum d'un nœud est égal à n .

Arbre binaire : c'est un arbre où le degré maximum d'un nœud est égal à 2.

Arbre binaire de recherche : c'est un arbre binaire où la clé de chaque nœud est supérieure à celles de ses descendants gauche, et inférieure à celles de ses descendants droits.

2. Arbres binaires

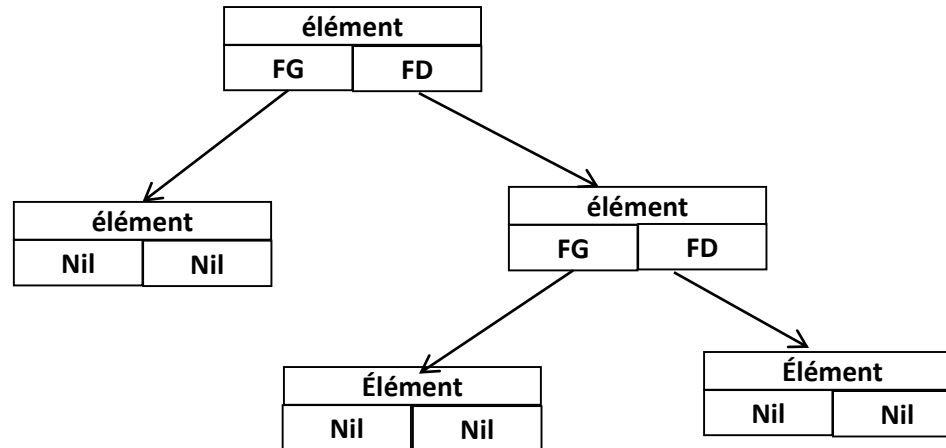
2. 1 définition

Soit un ensemble **de nœuds** auxquels sont associés des "valeurs" (les éléments à stocker : entier, réel, structure,...). Un arbre binaire est défini récursivement de la manière suivante : un arbre binaire est composé :

- de Nil (Arbre vide).
- Soit d'un seul nœud appelé **racine**,
- Soit d'un nœud **racine à la gauche** duquel est accroché **un sous-arbre binaire gauche**
- Soit d'un nœud **racine à la droite** duquel est accroché **un sous-arbre binaire droit**
- Soit d'un nœud racine auquel sont accrochés **un sous-arbre binaire droit et un sous-arbre binaire gauche**.

2.2 Implémentation

Représentation dynamique: utilisation d'une liste non linéaire, chaque nœud contient un élément avec l'adresse de son fils gauche (FG) et l'adresse de son fils droite (FD)



Définition de type

Type **Nœud** : Enregistrement

élément : Élément; // le type Élément signifie un type quelconque

FG: * **Nœud**; // pointeur vers le fils gauche

FD: * **Nœud**; // pointeur vers le fils droite

Fin ;

Type Arbre : * Nœud ;

2.3 opérations

1.créé_nœud crée un nœud contient la valeur x et sans fils ($FG=Nil$, $FD=Nil$) retourne un pointeur vers le nœud créé.

Fonction créé_nœud (x : Élément): Arbre

N:Arbre;

Début

allouer (N, Noued);

$N \rightarrow \text{élément} \leftarrow x$;

$N \rightarrow FD \leftarrow Nil$;

$N \rightarrow FG \leftarrow Nil$;

Retourne (N)

Fin

2.La fonction Contenu: retourne le contenu (la valeur) d'un nœud équivalent à premier pour les liste.

Fonction contenu (N: Arbre): Élément

Début

Retourne ($N \rightarrow \text{élément}$)

Fin

2.3 opérations

3. filsG (N: Arbre): retourne le fils gauche d'un arbre (le sous arbre gauche).

Fonction filsG (N: Arbre): Arbre

Début

Retourne (N->FG)

Fin

4. filsD (N: Arbre): retourne le fils droite d'un arbre (le sous arbre droite).

Fonction filsD (N: Arbre): Arbre

Début

Retourne (N->FD)

Fin

2.3 opérations

5. **Est_vide**: teste si un arbre est vide ou non.

Fonction Est_vide(a: Arbre): booléen

Début

 Retourne (a=Nil)

Fin

6. **Est_feuille** : teste si un nœud est une feuille ou non.

Fonction Est_feuille(N: Arbre): booléen

Début

 Retourne (N!=Nil **et** filsG(N)=Nil **et** filsD(N)=Nil)

 // ou retourne (non Est_vide(N) **et** Est_vide(filsG(N)) **et**
 Est_vide(FilsD(N)));

Fin

2.4 opérations de parcours

1. Parcours en profondeur: trois types

Préfixé : La règle de ce parcours est très simple : **la racine de l'arbre est traitée avant ses descendants.**

Procédure préfixe (a: arbre)

Début

```
Si a != Nil alors
    écrire (contenu (a));
    préfixe ( filsG(a)) ;
    préfixe ( filsD(a)) ;
```

Finsi

fin

Infixé : La règle de ce parcours est comme suit : **la racine de l'arbre est traitée entre ses descendants.**

Procédure Infixé(a: arbre)

Début

```
Si! est_vide(a) alors
    Infixé( filsG(a)) ;
    écrire (contenu (a));
    Infixé( filsD(a)) ;
```

finsi

fin

2.4 opérations de parcours

1. Parcours en profondeur: trois types

Préfixé : La règle de ce parcours est très simple : **la racine de l'arbre est traitée avant ses descendants.**

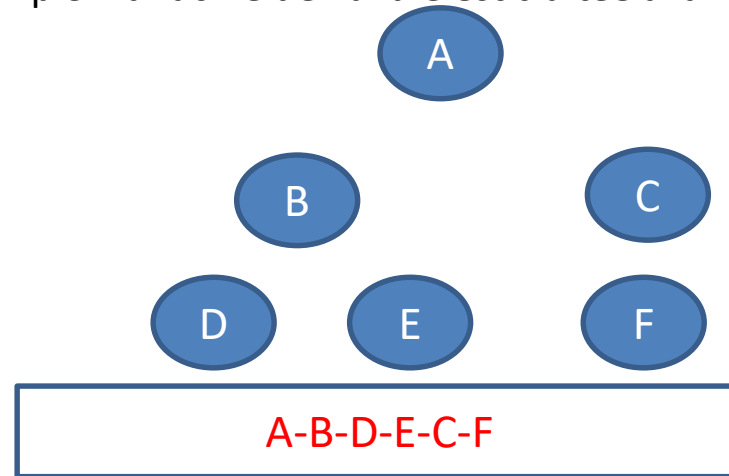
Procédure préfixe (a: arbre)

Début

```
Si a != Nil alors  
  écrire (contenu (a));  
  préfixe ( filsG(a)) ;  
  préfixe ( filsD(a)) ;
```

Finsi

fin



Infixé : La règle de ce parcours est comme suit : **la racine de l'arbre est traitée entre ses descendants.**

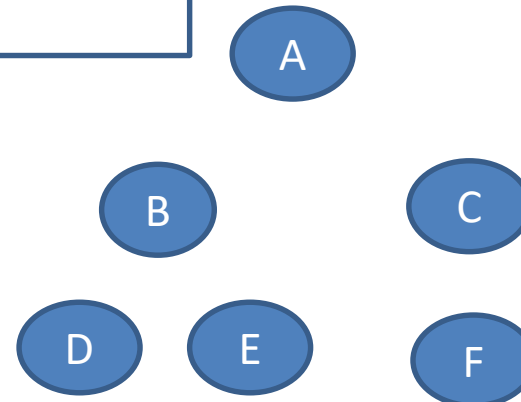
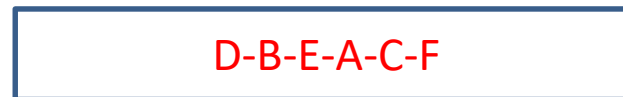
Procédure Infixé(a: arbre)

Début

```
Si! est_vide(a) alors  
  Infixé( filsG(a)) ;  
  écrire (contenu (a));  
  Infixé( filsD(a)) ;
```

finsi

fin



2.4 opérations de parcours 78 125 15 09

Postfixé : La règle de ce parcours est comme suit : **la racine de l'arbre est traitée après ses descendants.**

Procédure Postfixé (a: arbre)

Début

Si! est_vide(a) alors

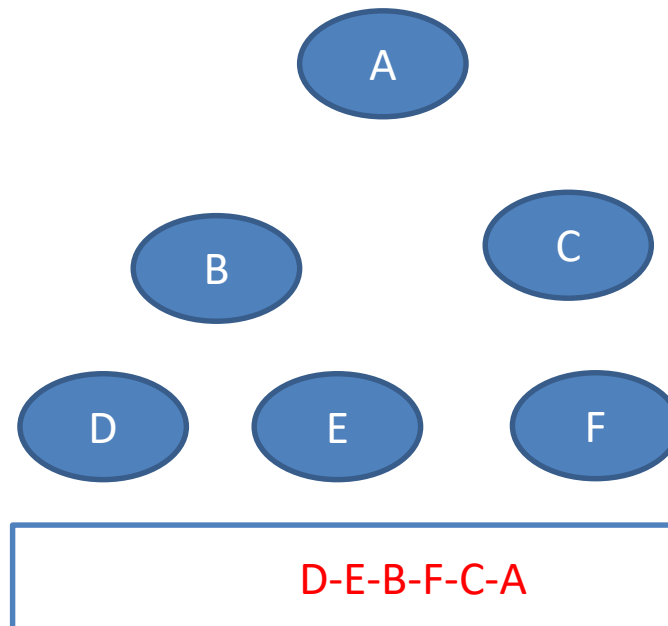
Postfixé (filsG(a)) ;

Postfixé (filsD(a)) ;

écrire (contenu (a));

finsi

fin



2. Parcours en largeur: Dans un parcours en largeur, tous les nœuds à une profondeur **i** doivent avoir été visités avant que le premier nœud à la profondeur **i + 1** ne soit visité. Un tel parcours nécessite que l'on se souvienne de l'ensemble des branches qu'il reste à visiter. Pour ce faire, on utilise une file d'attente.

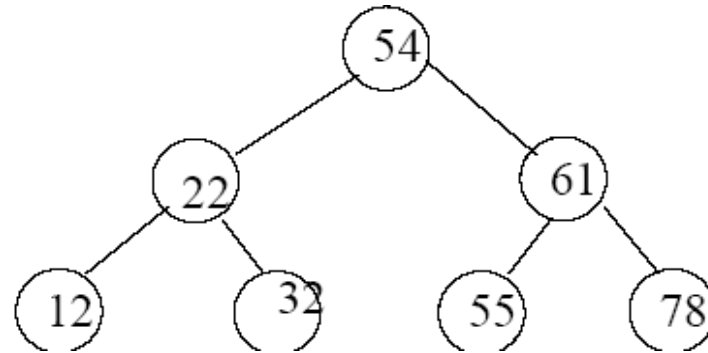
A-B-C-D-E-F

2.4 opérations de parcours

Exemple

Soit l'arbre A suivant:

A



Le parcours en largeur de l'arbre A donne: **54 22 61 12 32 55 78**

2.5 Recherche d'un éléments

1. appartient (x, a): Cette fonction teste si x existe dans l'arbre a ou non.

fonction appartient (type x, arbre a): booléen

Si est_vide (a)) alors

retourner faux;

Sinon

Si(contenu(a)= x) alors

retourner vrai;

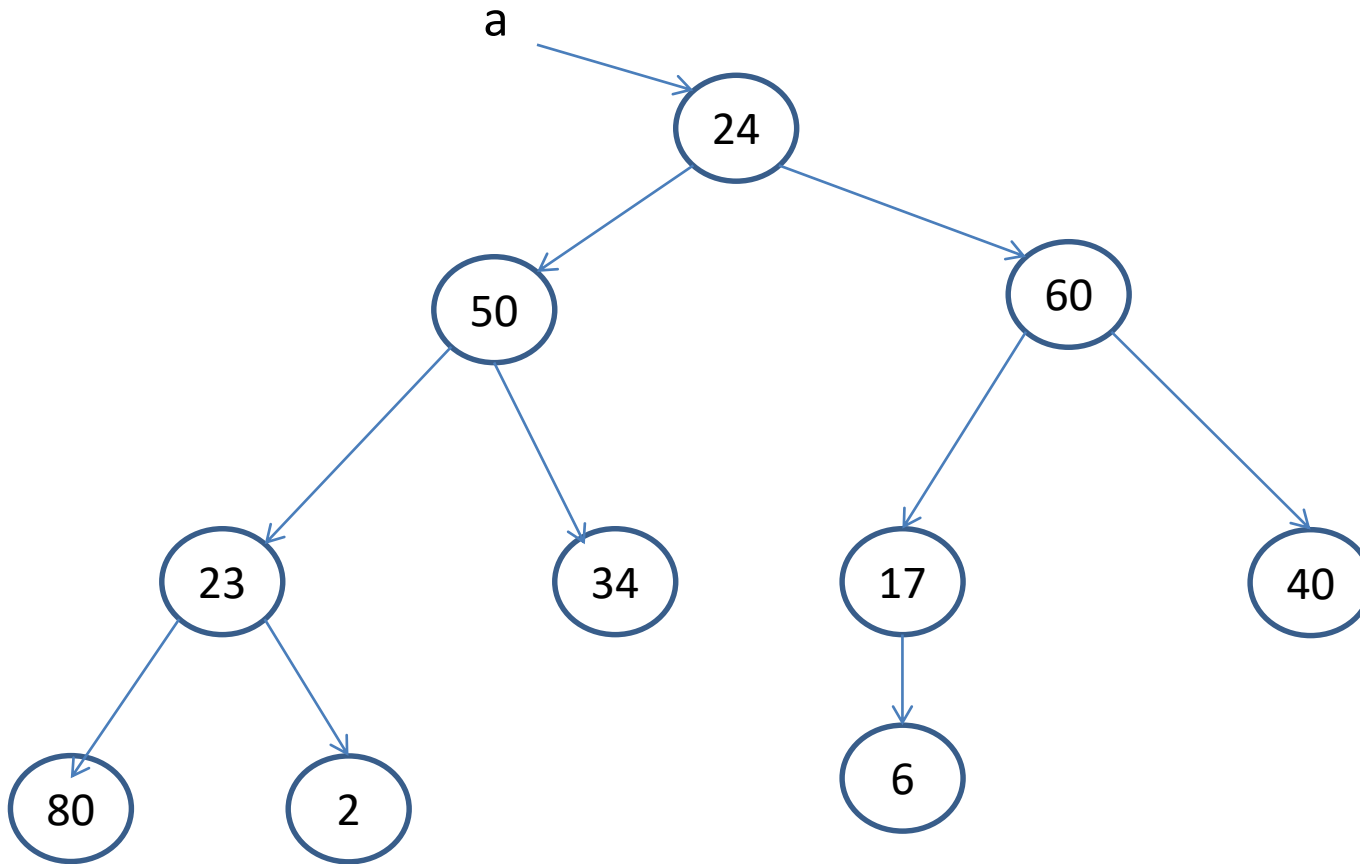
Sinon

retourner (appartienr (x, filsG(a)) ou appartient (x, filsD(a));

Finsi

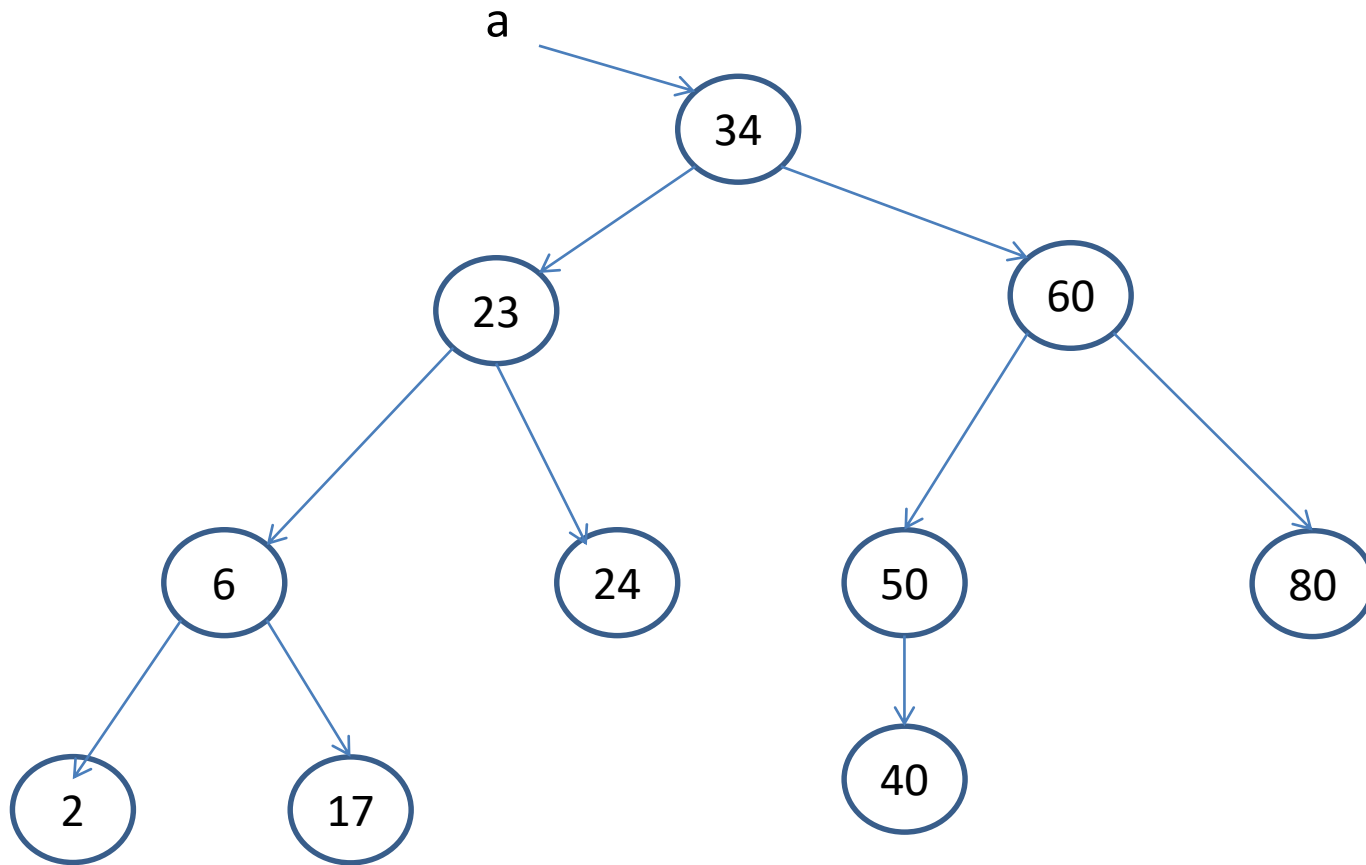
Finsi

Fin



Pour la recherche d'un éléments dans un arbre binaire nous devons parcourir tous les nœuds de l'arbre

3. Arbres binaires de recherche (ABR)

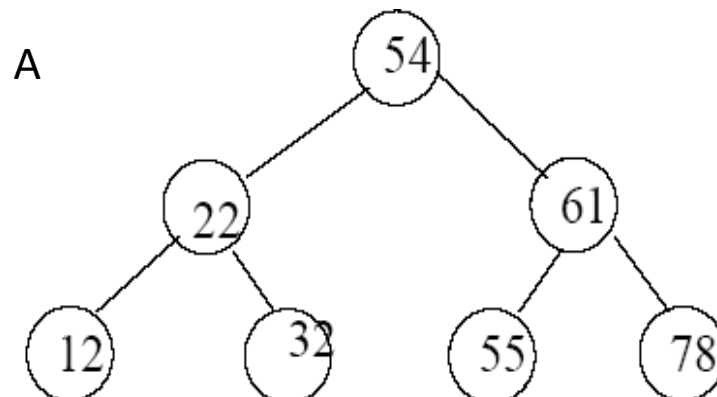


3.1 définition

Un arbre binaire de recherche est un arbre binaire vérifiant la propriété suivante :

soient x et y deux nœuds de l'arbre,

- si y est un nœud du sous-arbre gauche de x , alors $\text{clé}(y) < \text{clé}(x)$,
- si y est un nœud du sous-arbre droit de x , alors $\text{clé}(y) > \text{clé}(x)$.



3.2 Opérations

1. appartient (x, a): Cette fonction teste si x existe dans l'arbre a ou non.

fonction appartient (type x, arbre a): booléen

Si est_vide (a)) alors

retourner faux;

Sinon

Si (contenu(a) = x) alors

retourner vrai;

Sinon

Si (x < contenu(a)) alors

return (appartient (x, filsG(a)) ;

Sinon

return (appartient (x, filsD(a));

Finsi

Finsi

Finsi

Fin

3.2 Opérations

2. Insérer (x: Élément, a: Arbre): En utilisant le parcours préfixé, il sera très facile d'insérer de nouveaux nœuds.

Fonction **insérer** (a: arbre, x: type) arbre
début

Si est_vide (a)) **alors**

 a = crée_nœud(x);

Sinon

Si (x < contenu(a)) **alors**

 a -> FG = insérer (x, filsG(a)) ;

Sinon

 a -> FD = insérer (x, filsD(a));

Finsi

Finsi

 retourne a;

fin

3.2 Opérations

3.Suppression: La suppression d'une clé dans un arbre est une opération plus complexe. plusieurs cas sont à considérer selon le nombre de fils du nœud x:

- Si l'élément à supprimer n'existe pas on ne fait rien.
- Si l'élément à supprimer n'a pas de fils gauche, on le remplace par son fils droit.
- Si l'élément à supprimer n'a pas de fils droit, on le remplace par son fils gauche.
- Si l'élément à supprimer à deux fils, on le remplace par le plus grand (resp. petit) élément de son sous arbre gauche (resp. droit).

3.2 Opérations

3. Suppression: Trois fonction coopèrent pour la suppression d'un élément.

- La première, **supprimer**, recherche le nœud portant la clé à supprimer.

Fonction **supprimer**(a: arbre, x: type) arbre
début

Si ! est_vide (a)) alors

si (contenu(a) == x)

a = **supprimerRacine**(a);

sinon

Si (x < contenu(a))

a -> FG = **supprimer** (x, filsG(a)) ;

Sinon

a -> FD = **supprimer** (x, filsD(a));

Finsi

Finsi

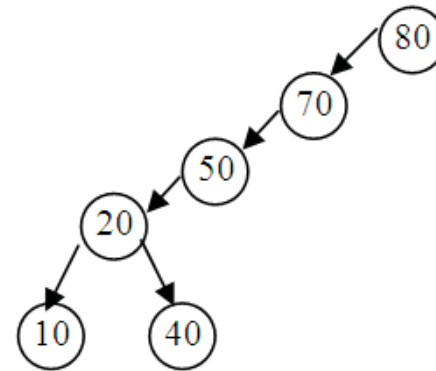
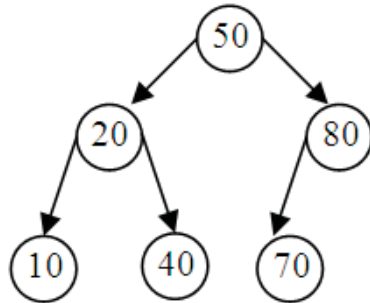
Finsi

retourne a;

Fin

3.3 Equilibrage

Soit les deux ABR suivants :



- Ces deux ABR contiennent les mêmes éléments, mais sont organisés différemment. La profondeur du premier est inférieure à celle du deuxième.
- Si on cherche l'élément 10 on devra parcourir 3 éléments (50, 20, 10) dans le premier arbre, par contre dans le deuxième, on devra parcourir 5 élément (80, 70, 50, 20, 10).
- On dit que le premier arbre est plus équilibré.

3.3 Equilibrage

- On dit qu'un ABR est équilibré si pour tout nœud de l'arbre la différence entre la hauteur du sous-arbre gauche et du sous-arbre droit est d'au plus égalé à 1. Il est conseillé toujours de travailler sur un arbre équilibré pour garantir une recherche la plus rapide possible.
- L'opération d'équilibrage peut être faite :
 - chaque fois qu'on insère un nouveau nœud,
 - chaque fois que le déséquilibre atteint un certain seuil pour éviter le coût de l'opération d'équilibrage qui nécessite une réorganisation de l'arbre.